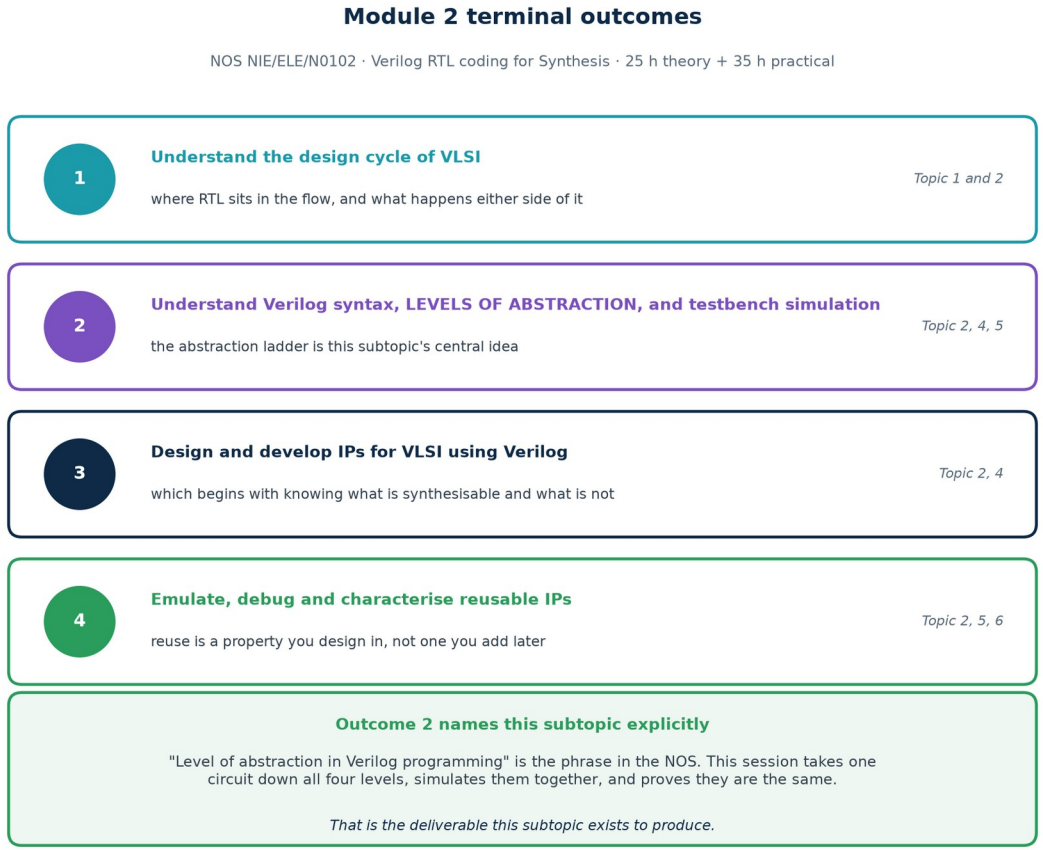
MODULE 2 · TOPIC 2

RTL Design Methodology

Basics of register transfer level (RTL) design · Overview of the RTL design process and methodology · Introduction to hardware description languages such as Verilog or VHDL

- Theory 1 · What RTL is — registers, transfers, and four levels of abstraction
- Theory 2 · The methodology — the flow, the synthesisable subset, the rules
- Theory 3 · HDLs — what one is, and Verilog against VHDL
- Practical · Labs A–I · 14 hours · 60 exercises · every number measured

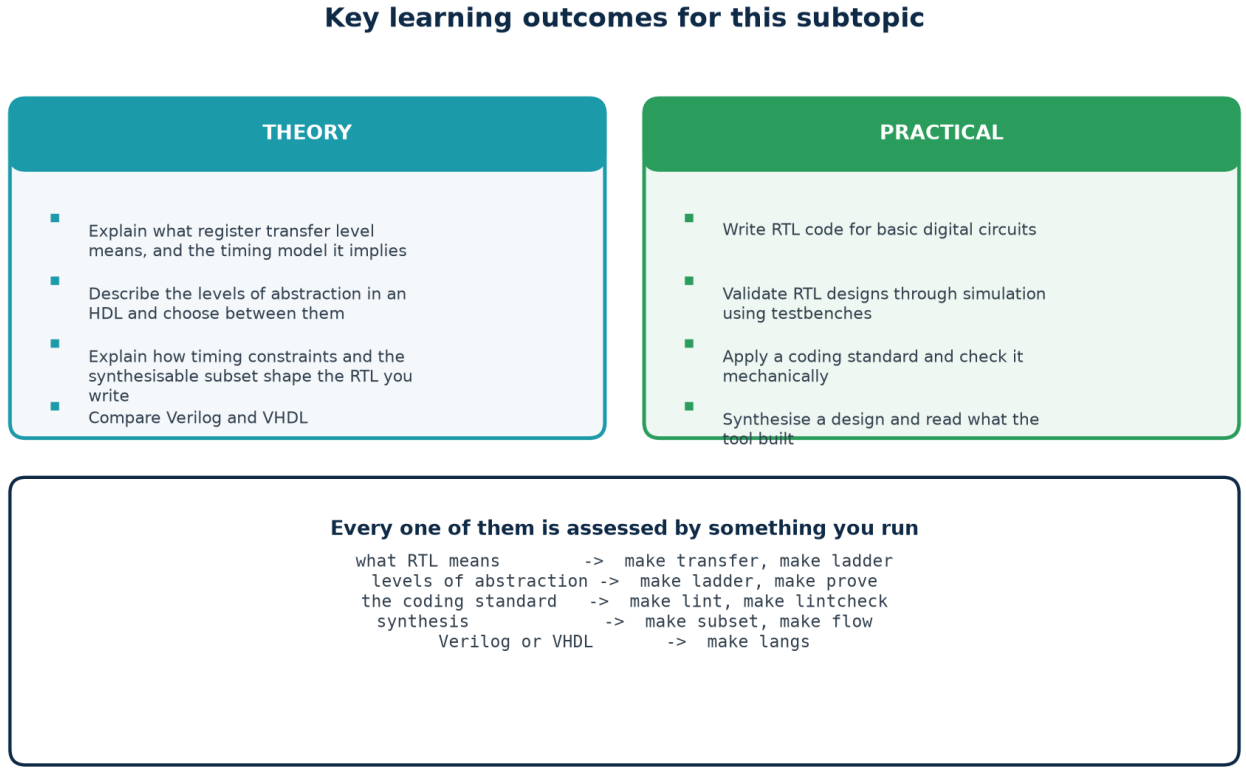
Terminal Outcomes



Outcome 2 names this subtopic in the NOS itself

"Level of abstraction in Verilog programming" is the phrase. This session takes one circuit down all four levels, simulates them together, and proves they are the same — which is the deliverable the outcome asks for.

Key Learning Outcomes



Every outcome is assessed by a command you run, not by something you recite.

Every Syllabus Phrase, and Where It Is Covered

Every syllabus phrase, and where it is covered

syllabus phrase	section	slides
Basics of register transfer level (RTL) design	**Theory 1**	5-16
Overview of RTL design process and methodology	**Theory 2**	17-38
Introduction to hardware description languages (HDLs) such as Verilog or VHDL	**Theory 3**	39-54
Practical: RTL Design and Implementation Labs	**Labs A-I**	55-70

The syllabus gives this subtopic 4 notional hours

Four hours is enough to define the terms. It is not enough to make anyone believe them, and nothing in this subtopic is believed until it has been seen to happen.

So the theory is developed to the depth the ideas need, and every claim is attached to a lab target that produces it. Deliver it in four hours by lecturing from the summary slides; deliver it properly by running the lab alongside.

On the four notional hours

Four hours is enough to define the terms. It is not enough to make anyone believe them — and nothing in this subtopic is believed until it has been seen to happen. Lecture from the summary slides to fit four hours; run the lab alongside to teach it.

How This Session Runs

RTL is not a language and it is not a tool. It is a way of thinking about hardware.

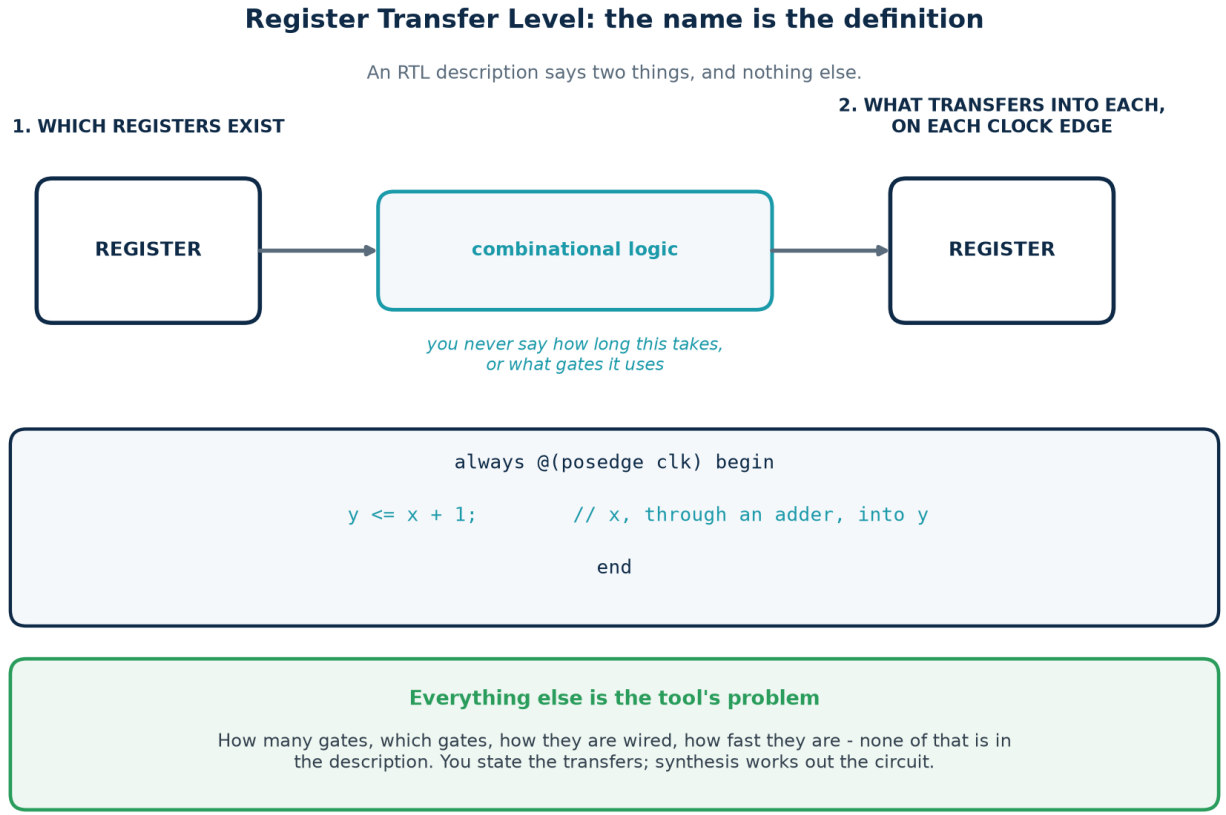
THEORY 1

What RTL Is

The name is the definition, and almost everything else in this topic is a consequence of it.

- Registers, and what transfers into them on each clock edge
- Why <= and = are not interchangeable
- Four levels of abstraction, one circuit
- And what a synthesiser makes of each level

Register Transfer Level: the Name Is the Definition



Two statements, and nothing else

1. Which registers exist. 2. What transfers into each one, on each clock edge.

How many gates, which gates, how they are wired, how fast they are — none of that is in the description. You state the transfers; synthesis works out the circuit.

Watch One Value Transfer

Watch one value transfer, one register per edge

A single 5 is applied on cycle 0 and never again.

cycle	din	x	y	z	acc
0	5	5	1	0	0
1	0	0	6	2	0
2	0	0	1	12	2
3	0	0	1	2	14
4	0	0	1	2	16

5 -> x one edge **6 -> y** one edge **12 -> z** one edge **-> acc**

Nothing moved between edges

The logic settles during the cycle and the registers all update together at the edge. That is the entire timing model of RTL, and it is why RTL is easy to reason about.

A single 5 applied on cycle 0. It reaches x, then y as 6, then z as 12, then acc — one register per clock edge, and nothing moves in between.

1.2 · WHY

Why Anyone Designs At This Level

	Behavioural / algorithmic	RTL	Gate netlist
you write	the algorithm	**registers and transfers**	every gate
timing	none at all	**one clock period per stage**	exact, per gate
synthesisable	rarely	**yes — this is the target**	yes, but why would you
a 10k-gate design	unbuildable	**a few hundred lines**	tens of thousands of lines
who writes it	architects, in C or SystemC	**you**	the synthesiser

RTL is the level where the trade lands correctly

High enough that a human can write and read a real design; low enough that a tool can build it without guessing at your intent.

Every industrial digital design in the last thirty years was written here. That is not fashion — it is where the abstraction pays for itself.

Above RTL you cannot say when things happen. Below it you cannot say anything else.

The Two Assignment Operators, and Why It Matters

Why clocked blocks use <= and combinational blocks use =

`<=` non-blocking

in CLOCKED blocks

every right-hand side is read first,
using the OLD values,
then every left-hand side updates
at the same instant

`a <= b;  b <= a;` swaps them.

`=` blocking

in COMBINATIONAL blocks

each statement completes before
the next one starts,
exactly the way software
statements run

`a = b;  b = a;` does not.

Get it backwards and the code still compiles

Blocking assignments in a clocked block make two blocks see each other's half-updated values, and which one wins depends on the order the simulator happens to run them in.

Non-blocking in a combinational block makes it behave like a register in simulation while synthesis builds plain logic - so the simulation and the silicon disagree.

Neither one is an error. Both are caught by rules L001 and L002 of the linter in this lab.

Neither one is an error. Both are caught by the linter you build in this topic.

The Swap, Worked Through

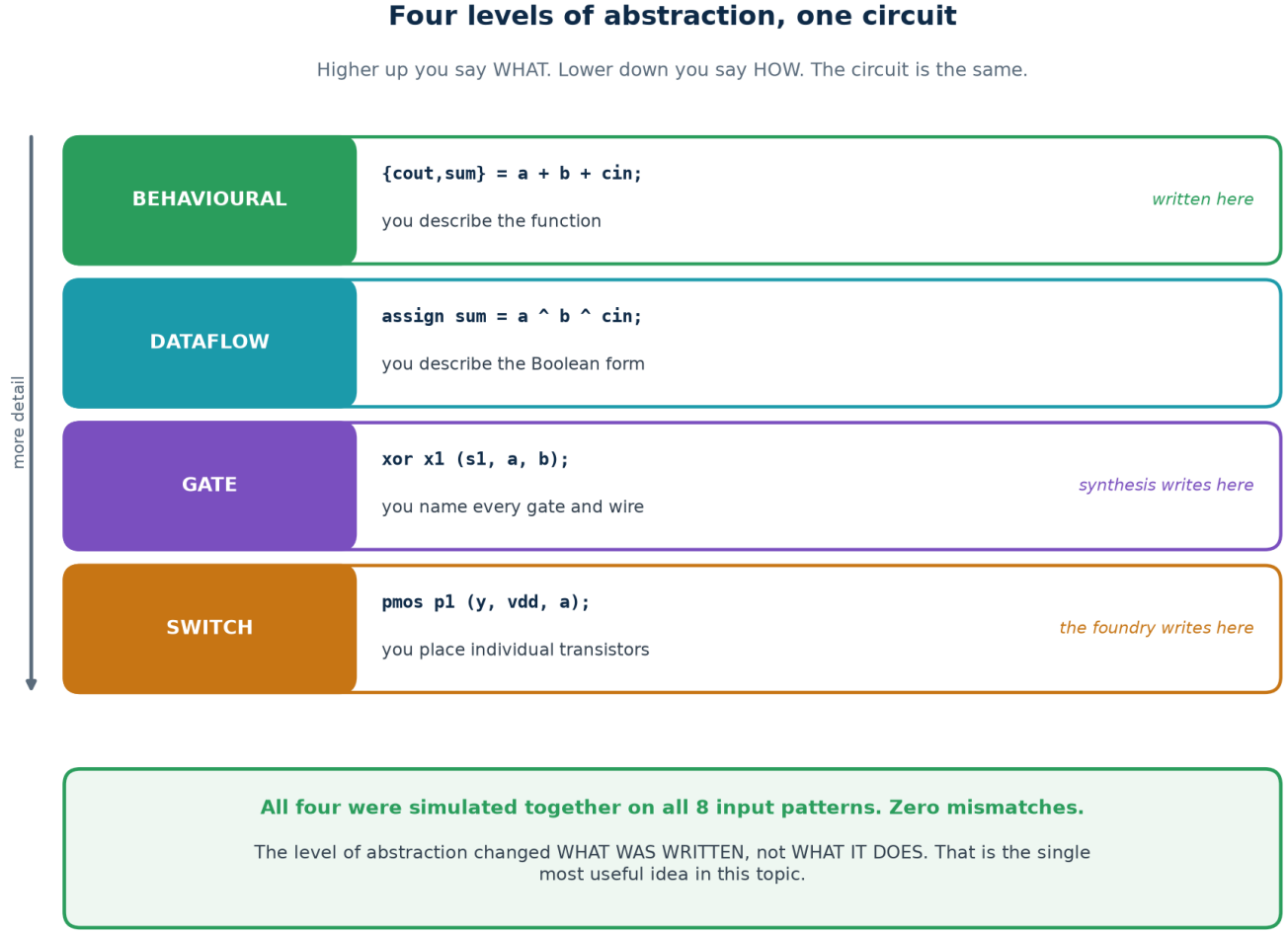
```
// NON-BLOCKING - inside always @(posedge clk)
//   Step 1: read every right-hand side, using the OLD values.
//   Step 2: update every left-hand side, all at the same instant.

        a <= b;        //  reads old b
        b <= a;        //  reads old a

        a and b are SWAPPED.  This is what a hardware register does.

// BLOCKING - inside always @*
//   Each statement finishes before the next one starts.

        a = b;         //  a is now b
        b = a;         //  a is already b, so this does nothing

        BOTH end up holding b.  This is what software does.
```

Why the rule is absolute rather than stylistic

Two clocked blocks using blocking assignments can see each other's half-updated values, and which one wins depends on the order the simulator happens to evaluate them in — an order the language standard deliberately does not fix.

Non-blocking assignment exists precisely to make that race impossible.

Four Levels, One Circuit



All four were simulated together on all 8 input patterns. Zero mismatches.

The Same Adder, Written Four Ways

```
// BEHAVIOURAL - you describe the function
always @* {cout, sum} = a + b + cin;

// DATAFLOW - you describe the Boolean form
assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);

// GATE - you name every gate and every wire
xor x1 (s1, a, b);      xor x2 (sum, s1, cin);
and a1 (ab, a, b);      and a2 (bc, b, cin);   and a3 (ac, a, cin);
or  o1 (cout, ab, bc, ac);

// SWITCH - you place individual transistors
pmos p1 (y, vdd, a);    nmos n1 (y, gnd, a);      // one CMOS inverter
```

And What a Synthesiser Makes of Each

And what a synthesiser makes of each level

written at	synthesis	cells	what came out
behavioural	OK	**5**	NAND x3, XOR x2
dataflow	OK	**6**	AND x2, OR x2, XNOR, ORNOT
gate	OK	**6**	identical netlist to dataflow
switch	REFUSED	**-**	transistors are not synthesisable

Two results worth stopping on

The BEHAVIOURAL description produced the SMALLEST circuit - five cells against six.

And dataflow and gate produced the IDENTICAL netlist: once you write the Boolean expression you have already chosen the structure, and naming the gates adds nothing but typing.

The rule this gives you

Write at the highest level that expresses your intent. Every level you descend takes a decision away from the tool and gives it to you - whether or not you wanted it.

The tool is better than you are at choosing gates. It is not better than you are at choosing architecture.

The behavioural description produced the SMALLEST circuit — and dataflow and gate produced the identical netlist.

1.4 · ABSTRACTION

The Rule This Gives You

Write at the highest level that expresses your intent

Every level you descend takes a decision away from the tool and gives it to you — whether or not you wanted it.

What the tool is better at than you

Choosing a gate mix for a Boolean function.

Balancing a logic tree.

Meeting a timing constraint by restructuring.

Doing all of that again, consistently, at 3 a.m.

What you are better at than the tool

How many clock cycles the job should take.

Where the registers go.

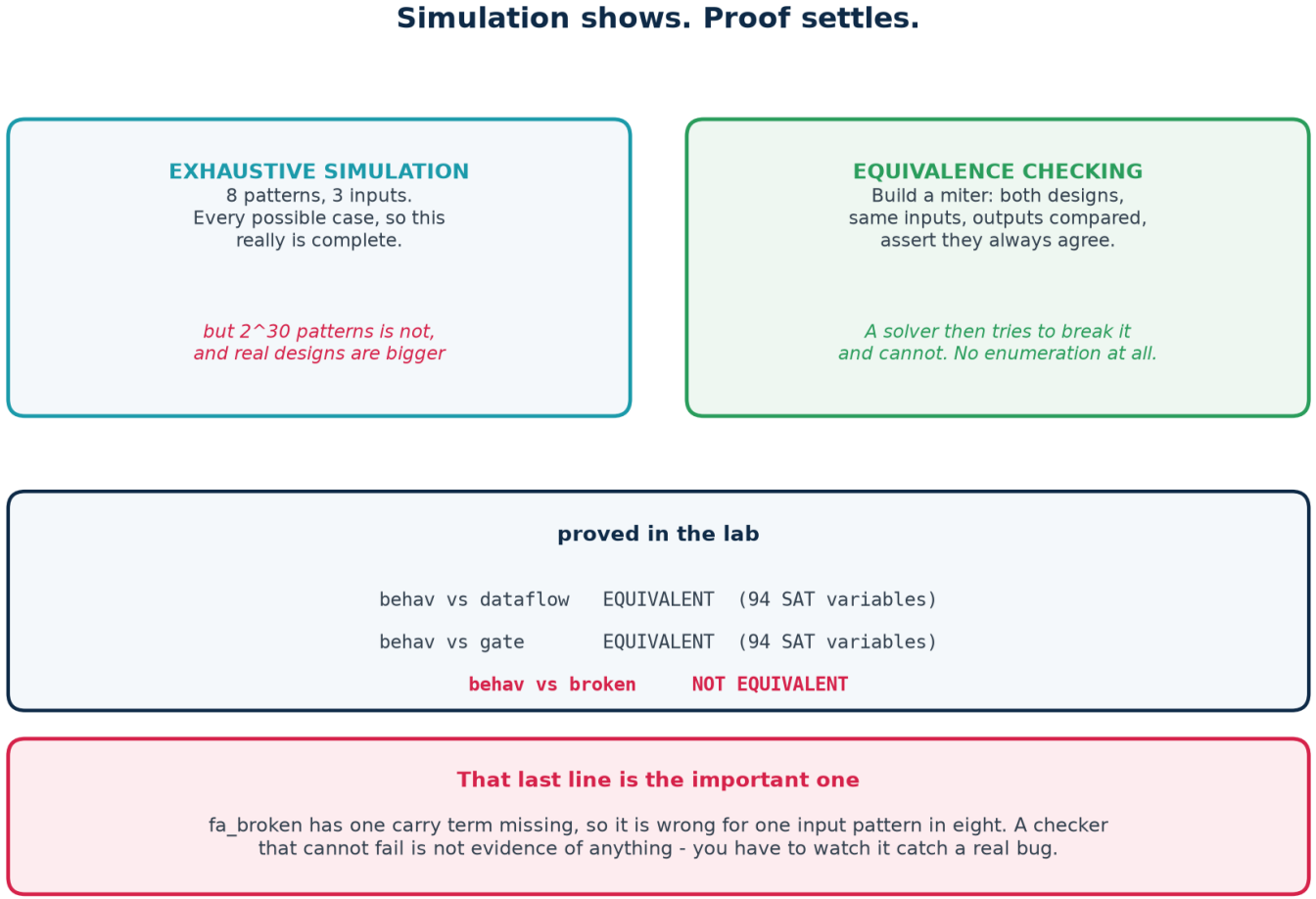
What is shared and what is duplicated.

What the interface should look like.

Give the tool the first list. Keep the second one for yourself.

1.5 · PROOF

Simulation Shows. Proof Settles.



A checker that cannot fail is not evidence of anything — which is why the lab includes a deliberately broken adder.

THEORY 1 · CHECKPOINT

Six Questions

#	Question	The answer in one line
1	What two things does an RTL description state?	which registers exist, and what transfers into them
2	What does <= mean inside a clocked block?	read all right-hand sides first, then update together
3	Name the four levels of abstraction.	behavioural, dataflow, gate, switch
4	Which level produced the smallest netlist, and why?	behavioural — it left the tool free to choose
5	Why did dataflow and gate give identical netlists?	writing the Boolean form already fixes the structure
6	Why prove equivalence instead of simulating?	exhaustive simulation stops being possible at ~30 inputs

Theory 2 asks what you are allowed to write, and why the rules exist.

THEORY 2

The RTL Design Process and Methodology

Methodology is the hardest thing to teach, because the honest version is a list of habits and the dishonest version is a list of slogans.

- The design flow, and where this topic sits in it
- The synthesisable subset — measured, not asserted
- The bug that makes simulation and silicon disagree
- Seven coding rules, and a tool that checks them
- Micro-architecture: the decisions RTL cannot make for you

The RTL Design Flow

The RTL design flow, and where this topic sits

SPECIFICATION	what it must do, in words, before any code	
MICRO-ARCHITECTURE	how many cycles, which registers, what partitioning	*Topic 2*
RTL CODING	the design, written in an HDL	*Topics 2, 4*
LINT	coding rules, checked mechanically - seconds	*Topic 2*
SIMULATION	does it do what the spec says?	*Topic 5*
SYNTHESIS	RTL becomes a gate netlist	*Topic 2, 4*
GATE-LEVEL CHECK	does the netlist still do it?	*Topic 2*
TIMING ANALYSIS	is it fast enough, and does it work at all?	*Topic 6*
PLACE AND ROUTE	gates become geometry	*Module 3*
SIGN-OFF	every check passes at every corner	*Module 3*

Every arrow points both ways in practice

A timing failure sends you back to the RTL; a synthesis surprise sends you back to the micro-architecture.

Every arrow points both ways in practice: a timing failure sends you back to the RTL, a synthesis surprise back to the micro-architecture.

Executed, Not Described

A methodology is a set of gates, not a set of suggestions

make flow - seven stages on one design, each producing evidence.

	stage	what runs	evidence produced
1	**SPEC**	written before the RTL	4 sentences
2	**LINT**	tools/rtl_lint.py	0 issues
3	**RTL SIMULATION**	iverilog, 18 cycles	wraps at 15, tc correct
4	**SYNTHESIS**	yosys	12 cells
5	**GATE SIMULATION**	the netlist, same stimulus	18 cycles
6	**COMPARE**	RTL against gate transcript	identical
7	**PROVE**	formal equivalence	proven by induction

Stage 7 is the one worth understanding

Stage 6 says the two agree on the 18 cycles that were tested. Stage 7 proves they agree on EVERY input sequence, by induction, without enumerating any of them.

The script stops at the first stage that fails. That is what makes it a methodology.

make flow stops at the first stage that fails. That is what makes it a methodology rather than a diagram.

Which Verilog Actually Synthesises

Which Verilog actually synthesises - measured

"Verilog is not a programming language" stays a slogan until you watch a tool refuse.

construct	synth	cells	what the tool said
always @* with full case	OK	**10**	clean
clocked block with <=	OK	**8**	8 flip-flops
if with no else	OK	**1**	inferred a LATCH
incomplete sensitivity list	OK	**1**	built anyway - see next slide
#5 delay in RTL	OK	**1**	the delay silently vanished
initial block	OK	**10**	accepted - FPGA only, not ASIC
for loop, constant bounds	OK	**7**	unrolled into 7 gates
while loop, data-dependent	REFUSED	**-**	"only allowed in constant functions"
real (floating point)	REFUSED	**-**	"syntax error, unexpected TOK_REAL"
a / b (variable divisor)	OK	**371**	a full combinational divider
a / 4 (constant divisor)	OK	**0**	no logic at all - just wires

371 cells against 0, for the same operator

The difference is entirely what you divided BY. A constant power of two is a rename of wires; a variable divisor is one of the largest things you can ask for by accident.

Eleven constructs, run through a real synthesiser. The table is measured, not remembered.

Three Rows Worth Discussing

371 cells against 0

a / b where b is a signal builds a full combinational divider — 371 cells.

a / 4 builds nothing at all. It is a rename of wires.

Same operator. The difference is entirely what you divided BY.

#5 vanished silently

A delay is a simulation instruction. Silicon has no way to wait five time units.

Synthesis ignored it and built the gate. Your simulation and your chip now behave differently.

initial was accepted

Yosys targets FPGAs, where the bitstream really does initialise the flops.

An ASIC flow would not. This is how code that works on an FPGA fails on an ASIC.

The habit this table is for

Before you write a construct you are unsure about, synthesise a ten-line module containing only that construct and read what came out. It takes two minutes and it is the only way to actually know.

The Most Common RTL Bug There Is

The inferred latch: the most common RTL bug there is

```
always @* begin
  if (en) y = d;
  end
```

there is no else, so you never said what y is when en = 0

so the tool must build something that REMEMBERS

$_DLATCH_P_

a level-sensitive latch you did not ask for

Why it is worse than an error

It is not an error at all. The tool builds it, mentions it in a log nobody reads, and hands you a design with a memory element in the middle of what you thought was combinational logic - which then has its own timing requirements, and breaks your static timing analysis.

It is not an error. The tool builds it, mentions it in a log nobody reads, and hands you the design.

Why a Latch In the Middle of Your Logic Hurts

	A flip-flop	A latch
captures	on the clock EDGE	while the enable is HIGH
is transparent	never	for the whole enable window
timing analysis	one clean check per edge	time-borrowing; much harder to analyse
in a scan chain	standard	usually needs special handling
you asked for it	yes, by writing posedge	no — you forgot an else

So the rule is mechanical

In a combinational block, assign every output on every path. An if needs an else; a case needs a default. If you would rather not write them out, assign a default value at the top of the block and let later assignments override it.

Rules L005 and L006 of the linter catch this, and make lintcheck confirms the linter agrees with Yosys on every file.

When Simulation and Silicon Disagree

When simulation and silicon disagree

```
always @(a) begin  y = a & b;  end        - b is read, but not in the list.
```

stimulus	a	b	RTL y	NETLIST y	
start	0	0	0	0	
change a	1	0	0	0	
change b	1	1	0	1	DISAGREE
change b	1	0	0	0	
change a	0	0	0	0	
change b	0	1	0	0	

Your testbench was verifying a circuit that will never be built

The RTL only re-evaluates when a changes, so y goes stale whenever b moves alone.
Synthesis ignored the sensitivity list entirely and built the AND gate you meant.

One disagreement in six - and nothing anywhere reported an error.

Which is why the rule is: never maintain a sensitivity list by hand

always @* builds it for you, and keeps it correct every time you edit the block.
SystemVerilog's always_comb does the same and also makes the tool check it.

One disagreement in six — and nothing anywhere reported an error.

Why This Class of Bug Is Special

Most bugs are found by testing. This one cannot be, because **the thing you are testing is not the thing that will be built**. Every test you write passes, and the chip still fails.

Cause	What simulation does	What synthesis does
incomplete sensitivity list	the block sleeps; the output goes stale	ignores the list; builds the logic
# delays in RTL	waits the specified time	ignores them entirely
initial block (ASIC)	sets the value at time zero	ignores it; the flop powers up unknown
blocking in a clocked block	result depends on evaluation order	builds one specific circuit

All four have the same cure

Use always @* and never a hand-written sensitivity list. No delays in RTL. A real reset instead of an initial block. Non-blocking in clocked blocks. Then lint for all four, so that nobody has to remember.

Seven Rules, Checked By a Tool

Seven rules, checked by a tool instead of by memory

rule	what it catches	why it matters
L001	blocking (=) in a clocked block	two blocks see half-updated values
L002	non-blocking (<=) in a combinational block	simulates like a register, synthesises as logic
L003	= and <= mixed in one block	no reader can tell the intent
L004	explicit sensitivity list	you have to maintain it, for ever
L005	if with no else in a combinational block	infers a LATCH
L006	case with no default	infers a LATCH
L007	signal driven from two always blocks	two drivers - last one to run wins

And the two latch rules are not opinions

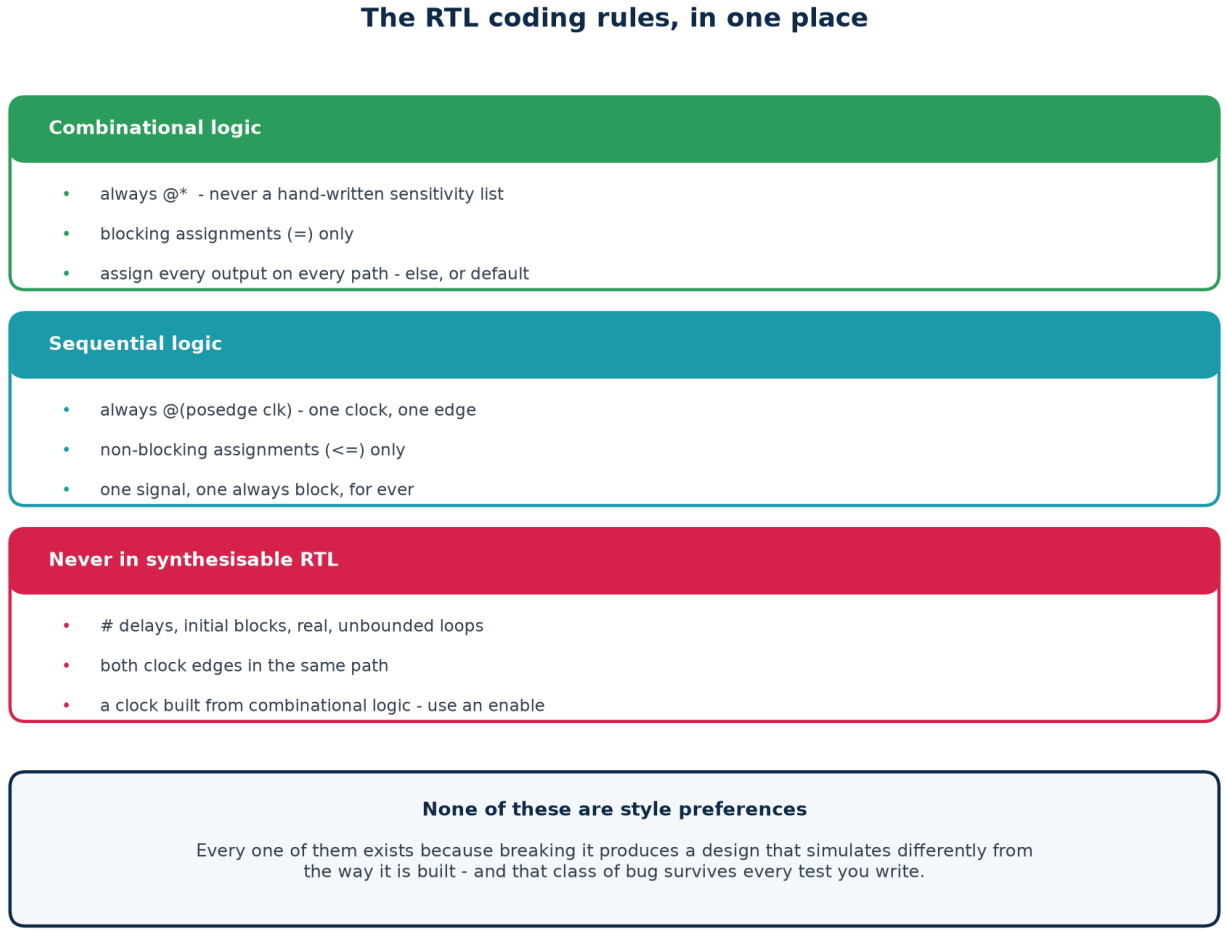
L005 and L006 claim a synthesiser will build a latch. That is a claim about what a tool does, so the tool settles it: make lintcheck runs both and compares.

`10 files · linter and Yosys agree on every one · 0 disagreements`

A linter that cries wolf gets switched off; one that stays quiet is worse than none at all.

A linter that cries wolf gets switched off. One that stays quiet is worse than none at all.

The Coding Standard, In One Place



None of these are style preferences. Every one exists because breaking it produces a design that simulates differently from the way it is built.

The Decisions RTL Cannot Make For You

Micro-architecture: the decisions RTL cannot make for you

Synthesis chooses gates. It does not choose any of these.

the decision	the options	what it costs you
How many clock cycles?	one big cycle, or a pipeline	Fmax, latency, area
Where do the registers go?	stage boundaries	Fmax, and directly
What is shared?	one multiplier reused, or four in parallel	area against throughput
How is it partitioned?	module boundaries, hierarchy	readability, reuse, synthesis runtime
What is the interface?	handshake, valid/ready, fixed latency	how easily it plugs into anything else
Where does reset reach?	everything, or only what needs it	area, and routing congestion

This is where the engineering is

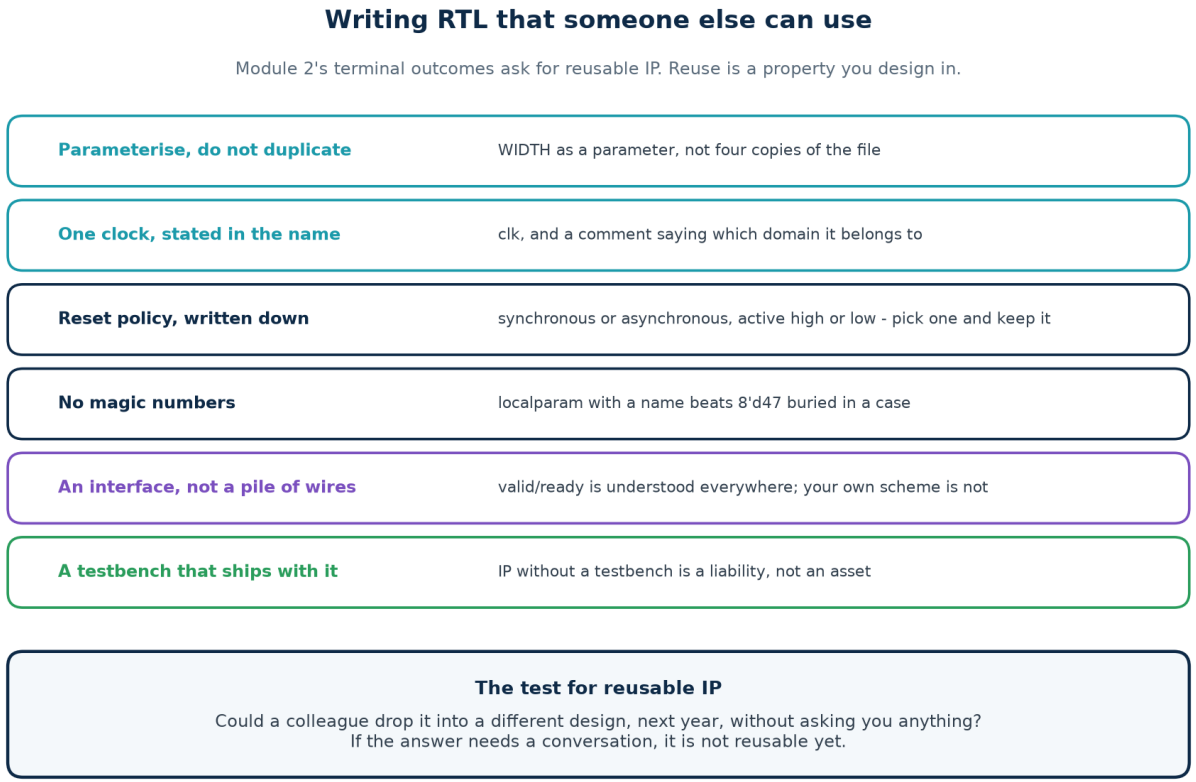
A synthesiser is better than you are at choosing gates. It is not better than you are at any of the rows above, and it will not warn you that you chose badly - it will faithfully build what you asked for.

Make these decisions on paper, before the first line of RTL.

Make these on paper, before the first line of RTL. A synthesiser will faithfully build whatever you chose.

2.7 · REUSE

Writing RTL That Someone Else Can Use



Module 2's fourth terminal outcome asks for this by name

"Emulate, debug and characterise reusable IPs." Reuse is not something you add to a design afterwards — it is a set of decisions you take while writing it, and every one of them is listed above.

THEORY 2 · CHECKPOINT

Seven Questions

#	Question	The answer in one line
1	Name the first four stages of the flow.	spec, micro-architecture, RTL coding, lint
2	Why lint before simulating?	it costs seconds and catches what tests cannot
3	What builds a latch?	a combinational block that does not assign on every path
4	a / b against a / 4 — what did they cost?	371 cells against 0
5	Why is an incomplete sensitivity list so dangerous?	simulation and synthesis then build different circuits
6	What does # do in synthesisable RTL?	nothing at all — it is silently ignored
7	Name three decisions synthesis will not make for you.	cycle count, register placement, sharing, interface

Theory 3 is about the notation — which turns out to be the least important part.

Introduction to Hardware Description Languages

The syllabus says "HDLs such as Verilog or VHDL". The word doing the work is OR.

- Why an HDL is not a programming language
- Everything happens at once
- The parts of a module, and the word reg that confuses everyone
- Verilog and VHDL, side by side and both actually run

An HDL Is Not a Programming Language

An HDL is not a programming language

It looks like one. That resemblance is the single biggest source of beginner bugs.

A PROGRAM

- statements run one after another
- a variable holds a value
- a loop repeats over time
- a function is called and returns
- you describe a PROCEDURE

AN HDL DESCRIPTION

- every block runs ALL THE TIME
- a signal is a wire with a value on it
- a loop is unrolled into hardware
- a module is INSTANTIATED and exists
- you describe a STRUCTURE

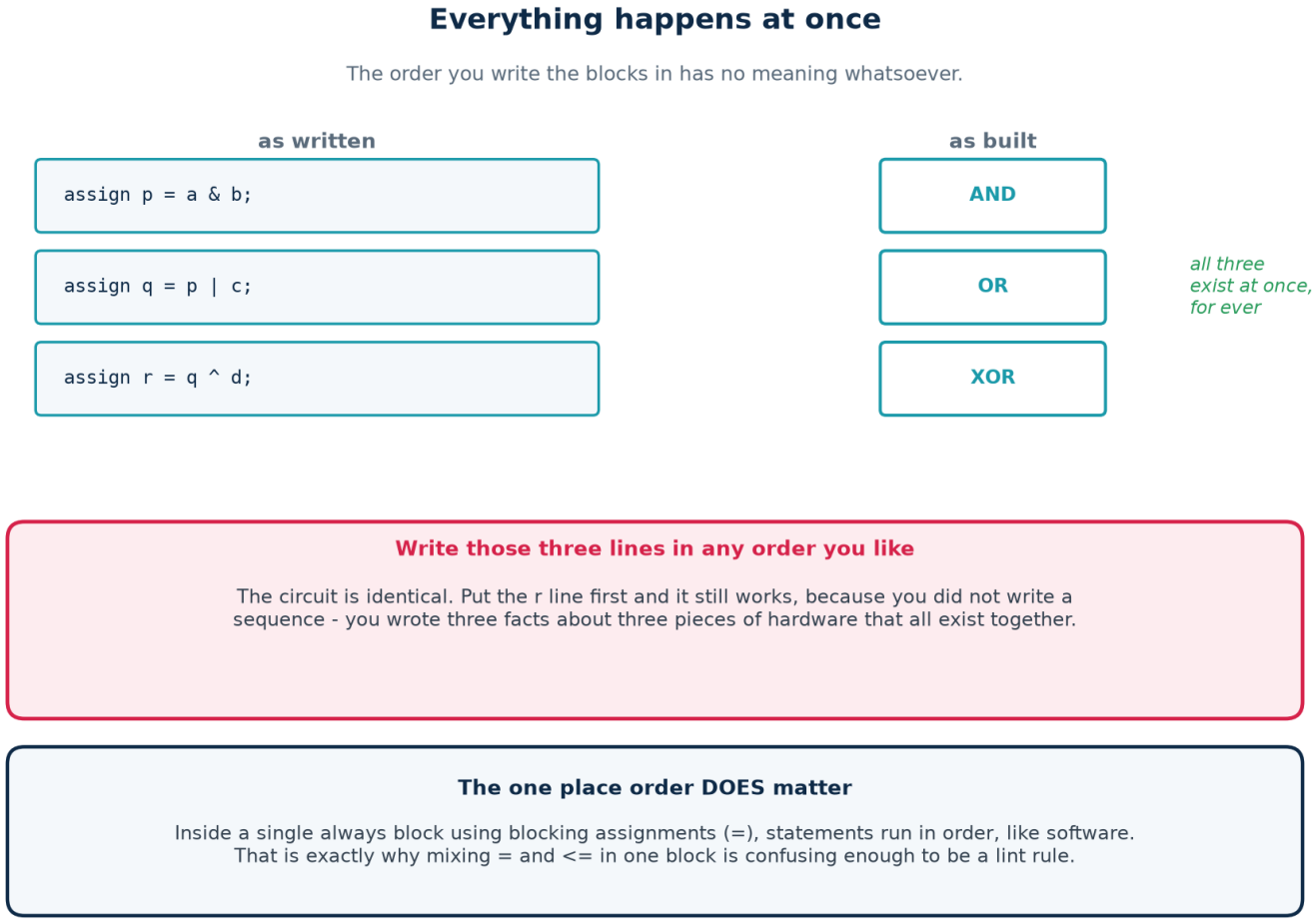
The sentence to keep in your head

You are not writing instructions for a machine to follow. You are writing a DESCRIPTION of a machine, which a tool will then build. The text is a blueprint, not a recipe.

Which is why "it compiles" means almost nothing, and why a construct can be perfectly legal Verilog and still have no hardware meaning at all.

You are not writing instructions for a machine to follow. You are writing a description of a machine.

Everything Happens At Once



Write those three assign lines in any order you like — the circuit is identical.

The Parts of a Verilog Module

The parts of a Verilog module

```
module counter #(parameter WIDTH = 4) (          // name, and parameters that let it be reused at other sizes
    input                 clk,                   // ports: the interface
    input                 rst,
    output reg [WIDTH-1:0] count                 // reg here means 'assigned in an always block', NOT a flip-flop
);

    always @(posedge clk) begin                  // a clocked block: this is where flip-flops come from
        if (rst) count <= 0;
        else     count <= count + 1;
    end

endmodule
```

The word that confuses everyone: reg

reg does NOT mean register. It means "this signal is assigned inside a procedural block".
A reg assigned in an always @* block becomes pure combinational logic.

What actually creates a flip-flop is assigning inside always @(posedge clk) - nothing else.
SystemVerilog renamed reg to logic precisely to end this confusion.

reg does not mean register. What creates a flip-flop is assigning inside always @(posedge clk) — nothing else.

How a Simulator Runs an HDL

The order blocks are evaluated in is genuinely unspecified. That is why non-blocking assignment exists.

Verilog and VHDL, Side By Side

Verilog and VHDL: the same ideas, different notation

	Verilog	VHDL
origin	1984, Gateway Design Automation	1983, US Department of Defense
standard	IEEE 1364, then 1800 (SystemVerilog)	IEEE 1076
feel	terse, C-like	verbose, Ada-like
typing	weak - it will let you do most things	strong - explicit conversions required
case sensitive	yes	no
interface / body	one module	entity and architecture, separately
a clocked block	always @(posedge clk)	process (clk) ... if rising_edge(clk)
assignment	<= and =	<= and :=
used most in	North America, ASIC flows	Europe, defence, aerospace

None of those differences are about hardware
Both describe registers, combinational logic and hierarchy. Both synthesise to the same gates. An engineer who understands RTL can read the other one after an afternoon; an engineer who has only memorised syntax can read neither.

The syllabus says "Verilog or VHDL". The word doing the work is OR.

None of those differences are about hardware. Both describe registers, logic and hierarchy; both synthesise to the same gates.

The Same Counter, In Both, Actually Run

The same counter, in both, actually run

Verilog · iverilog

```
always @(posedge clk)
  if (rst) count <= 0;
  else if (en)
    count <= count + 1;
assign tc = &count;
```

VHDL · ghdl

```
if rising_edge(clk) then
  if rst = '1' then
    cnt <= (others => '0');
  elsif en = '1' then
    cnt <= cnt + 1;
```

Two languages, two simulators, transcripts diffed

```
cycle 14  count=1111  tc=1        cycle 15  count=0000  tc=0
```

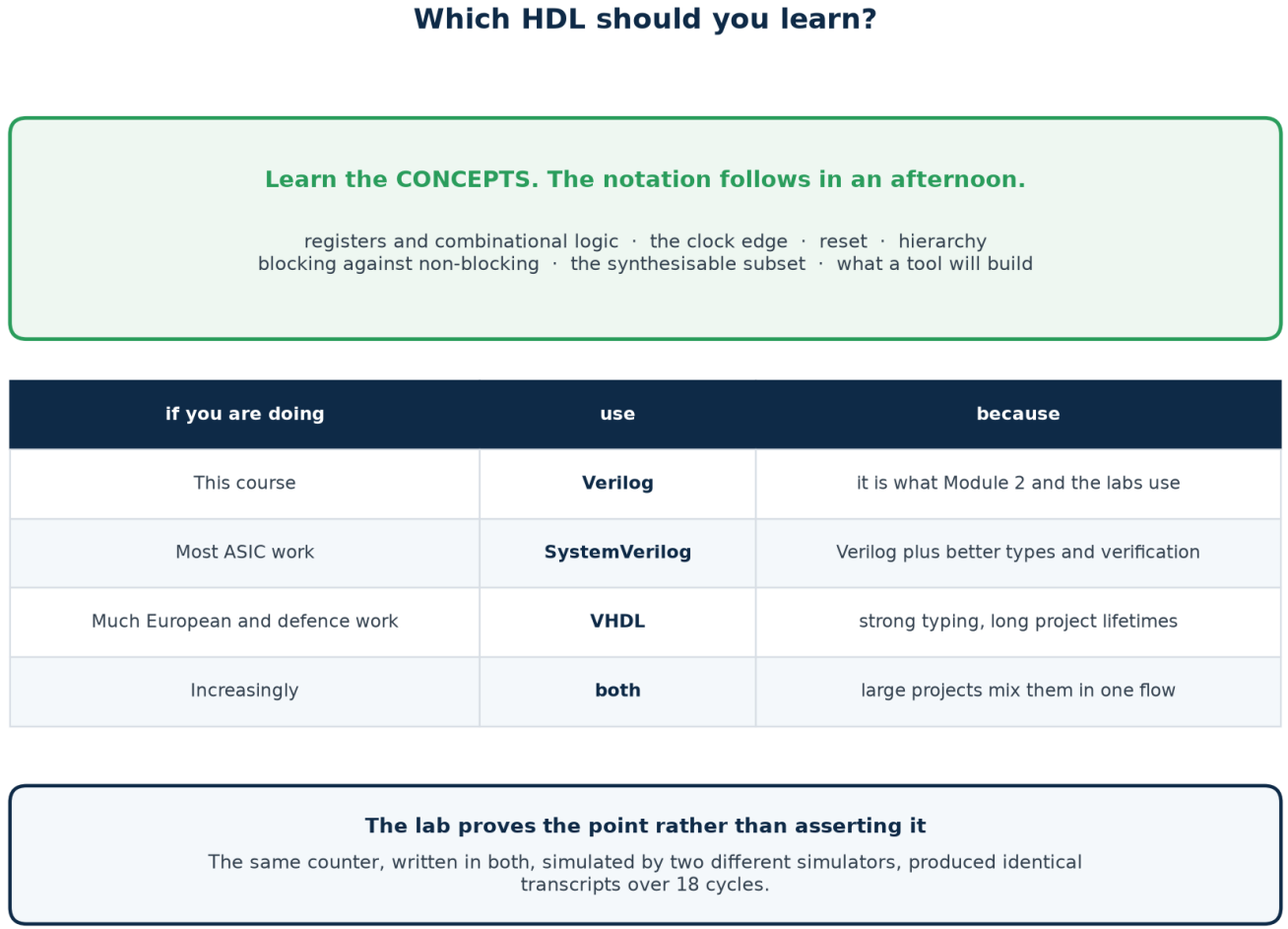
IDENTICAL over all 18 cycles - including the wrap and the terminal count.

Not "they look similar". The two logs were compared line by line by diff, and there was nothing to report.

Not "they look similar"

The two transcripts were compared line by line by diff, and there was nothing to report — 18 cycles, two languages, two different simulators, one design.

Which HDL Should You Learn?



Learn the concepts. The notation follows in an afternoon.

THEORY 3 · CHECKPOINT

Five Questions

#	Question	The answer in one line
1	Name the biggest difference between an HDL and a program.	everything in an HDL runs all the time, concurrently
2	Does the order of assign statements matter?	no — they describe hardware that exists together
3	What does reg actually mean?	assigned in a procedural block; NOT a flip-flop
4	Why is the evaluation order unspecified?	the standard leaves it free; <= makes it not matter
5	Verilog or VHDL?	either — the concepts are identical, and the lab proves it

PRACTICAL

Labs A–I

14 hours, 60 graded exercises, and not one number in this deck that you cannot reproduce yourself.

- Which tool answers which question, and how to install them
- Labs A–C: what RTL means, the ladder, and proof
- Labs D–F: the subset, the mismatch, and the coding rules
- Labs G–I: two languages, the whole flow, and the vendor tools

PRACTICAL · TOOLS

The Tools, and What Each One Is For

The tools, and what each one is for

tool	licence	what it does	used in
Icarus Verilog	free	simulate Verilog	this lab
GHDL	free	simulate VHDL	this lab
GTKWave	free	look at the waveforms	this lab
Yosys	free	synthesise, and prove equivalence	this lab
Verilator	free	fast simulation, and a good linter	recommended
rtl_lint.py	you write it	the seven coding rules	this lab
Vivado	free WebPACK	synthesis and implementation, Xilinx	syllabus tool
ModelSim / Questa	free starter	simulation, both languages	syllabus tool
Design Compiler	commercial	ASIC synthesis	industry

Everything in this topic runs on the free rows

Yosys is the interesting one: it is the only free tool here that will both synthesise your RTL and PROVE that the netlist it produced is equivalent to what you wrote.

Vivado and ModelSim are named by the syllabus and are covered in the deck; they were not used to produce any number in these materials.

Yosys is the interesting one: the only free tool here that both synthesises your RTL and PROVES the netlist matches it.

PRACTICAL · INSTALL

Two apt Lines, and the Whole Lab Runs

Install this, and the whole lab runs

Two apt lines. No licence, no account, no download manager.

REQUIRED — *Debian / Ubuntu / WSL2*

```
sudo apt update

sudo apt install yosys iverilog gtkwave python3

sudo apt install ghdl          # only for the VHDL comparison
```

Icarus — simulates Verilog - and the transistor level too

GHDL — simulates VHDL, so 'Verilog or VHDL' can be shown

Yosys — synthesises, and proves equivalence with a SAT solver

GTKWave — for when the transcript is not enough

```
yosys -V  &&  iverilog -V  &&  ghdl --version
```

Three version strings and every target in the lab Makefile will run.
Without ghdl, everything except make langs still works.

Debian / Ubuntu / WSL2

```
sudo apt install yosys iverilog gtkwave python3
sudo apt install ghdl                    # only for the VHDL comparison
yosys -V && iverilog -V && ghdl --version
```

Without ghdl everything except make langs still runs.

PRACTICAL · INSTALL

The Vendor Tools the Syllabus Names

The vendor tools the syllabus names

	what to do	watch out for
1	Create a free AMD/Xilinx account	the download will not start without one
2	Download the Unified Installer (~200 MB web installer)	not the 40 GB full archive
3	Choose Vivado ML Edition, then ML Standard	Standard is free; no licence file needed
4	Deselect every device family but the one you use	cuts 40 GB to about 8 GB
5	ModelSim: Intel FPGA Starter Edition is free	simulates both Verilog and VHDL
6	Verify: vivado -version and vsim -version	both then run headless

What these add, and what they do not

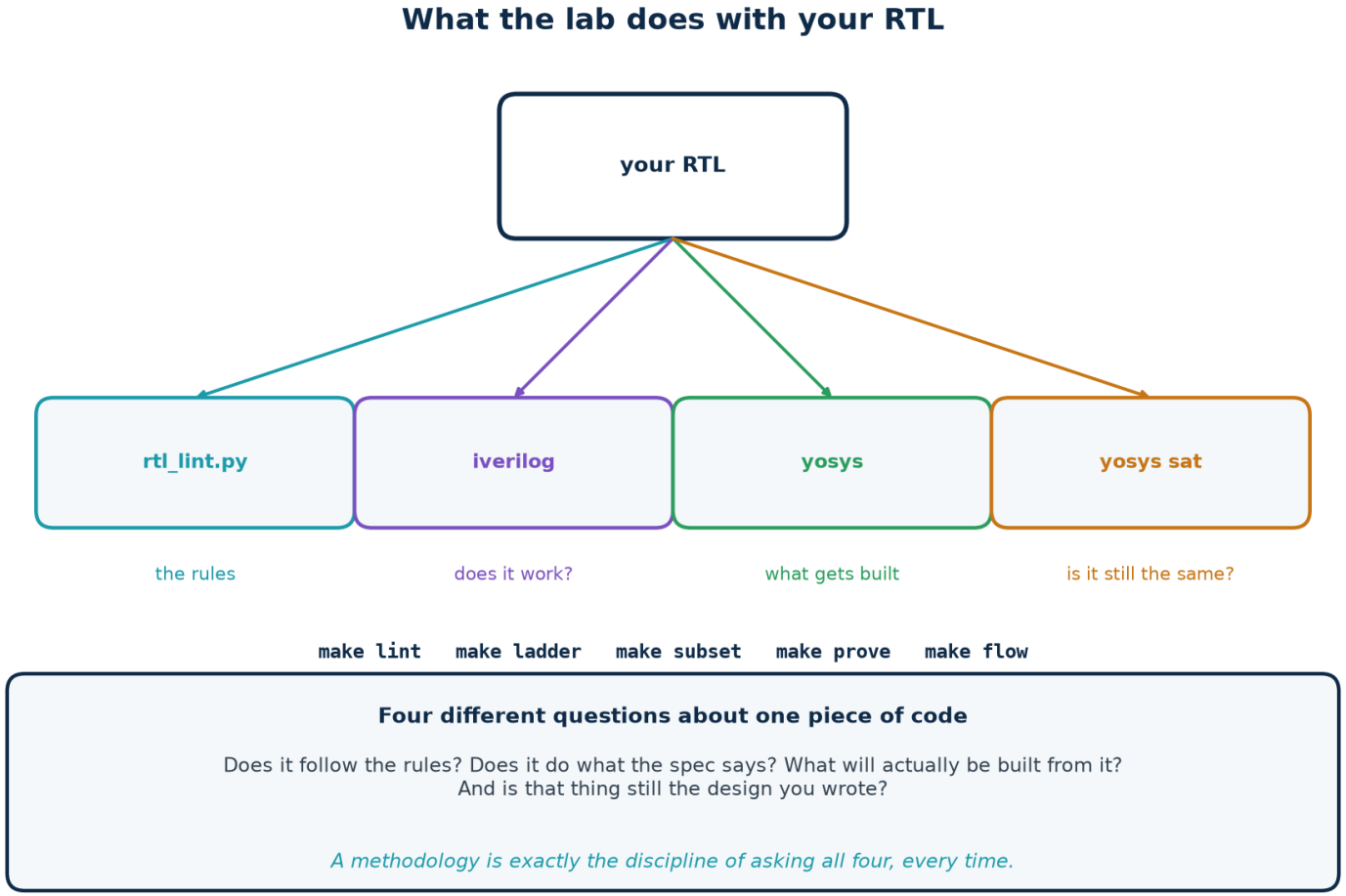
They add a real device library, real timing numbers, and a flow you will meet at work. They do not add anything to the CONCEPTS in this topic - RTL is RTL, and the free toolchain shows all of it.

Learn the ideas on the free tools, where the whole flow takes seconds. Then run the same design through Vivado and recognise every stage.

They add a real device library and a flow you will meet at work. They add nothing to the concepts.

PRACTICAL · THE FLOW

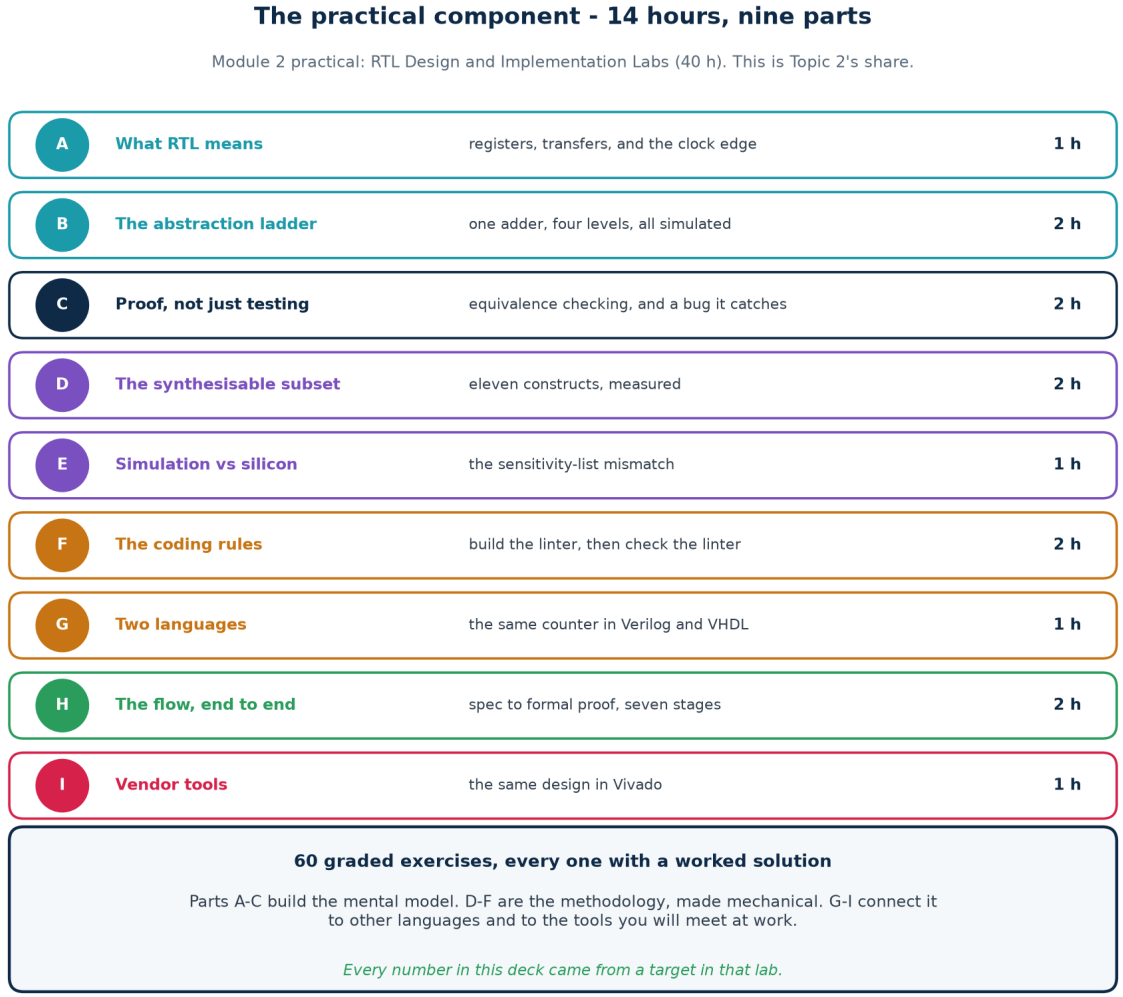
Four Questions About One Piece of Code



Does it follow the rules? Does it do what the spec says? What will be built? Is that still your design?

PRACTICAL · MAP

Nine Parts, Fourteen Hours



Parts A–C build the mental model, D–F the methodology, G–I the connection to other languages and real tools.

LABS A–C

What RTL Means, the Ladder, and Proof

A · what RTL means (1 h)

Run make transfer. Follow a single 5 through four registers.

Predict the value in each register on each cycle BEFORE running it.

Deliverable: the table, and why y is 1 on cycle 0.

B · the ladder (2 h)

Write the same full adder at all four levels.

Run make ladder — all four together, exhaustively.

Then synthesise each and compare cell counts.

Deliverable: why behavioural won.

C · proof (2 h)

Run make prove. Three pairs proved equivalent.

Then break the adder yourself and watch the checker catch it.

Deliverable: why 8 patterns was enough here and will not be next time.

The exercise that teaches the most is C

Delete a different term from fa_broken.v and predict which input pattern the solver will find. Getting that prediction right means you understand both the circuit and the tool.

LABS D–F

The Subset, the Mismatch, and the Rules

```
$ make subset
  s03_latch           OK       1     YES     inferred a LATCH: $_DLATCH_P_
  s08_whileloop       REFUSED  -     -       "only allowed in constant functions"
  s10_divide          OK       371   no      a full combinational divider
  s11_shift           OK       0     no      no logic at all - just wires

$ make mismatch
  change b (list ASLEEP)   a=1 b=1   RTL y=0   NETLIST y=1  <-- THEY DISAGREE

$ make lintcheck
  10 files  ·  linter and Yosys agree on every one  ·  0 disagreements
```

Exercise	Task
D.1	Predict the cell count for each construct before running make subset
D.2	Add a twelfth construct of your own and predict the result
E.1	Explain the mismatch to someone who has not seen it, in three sentences
F.1	Add rule L008 to the linter — your choice — and justify it
F.2	Find a file the linter gets wrong, and say why regexes cannot fix it

LABS G–I

Two Languages, the Whole Flow, and the Vendor Tools

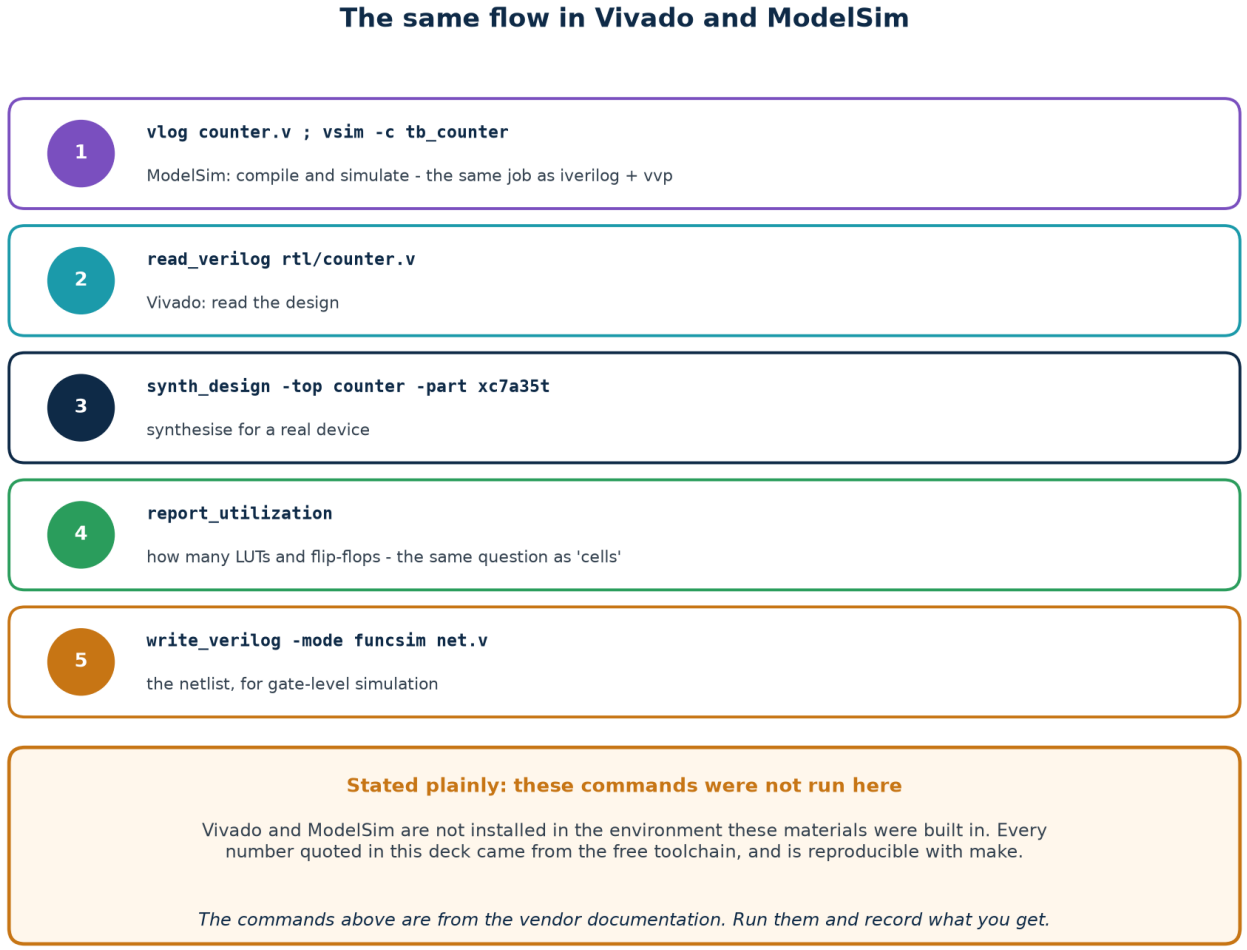
```
$ make langs
  === Verilog: iverilog ===        === VHDL: ghdl ===
  IDENTICAL over all 18 cycles - including the wrap and the terminal count.

$ make flow
  STAGE 2  LINT              0 issues
  STAGE 4  SYNTHESIS         12 cells
  STAGE 6  COMPARE           IDENTICAL on all 18 cycles
  STAGE 7  PROVE             Equivalence PROVEN by induction

  All seven stages passed.
```

Lab I: the same design in Vivado

Read the design, synthesise for a real part, report utilisation, write the netlist for gate-level simulation. Vivado is not installed in the environment these materials were built in, so lab I is the one whose output is not reproduced here — run it and record what you get.

PRACTICAL · VENDOR

The Same Flow in Vivado and ModelSim



Stated plainly on the slide: these commands were not run here. Every number in this deck came from the free toolchain.

LABS · ASSESSMENT

How the 60 Exercises Are Weighted

How the 60 exercises are weighted

part	exercises	weight	assessed on
A · what RTL means	6	10%	can you read a transfer as hardware
B · the ladder	9	15%	same circuit, four notations
C · proof	7	12%	why exhaustive testing stops working
D · the subset	10	18%	prediction before measurement
E · sim vs silicon	5	10%	explaining the mismatch precisely
F · coding rules	10	17%	the rule, and the reason for it
G · two languages	5	8%	reading VHDL without panic
H · the flow	8	10%	evidence at every stage

The rule that runs through every part

Predict, then measure. An exercise where you ran the command and wrote down the answer earns very little; one where you wrote the prediction first and then explained the difference earns it all.

Being wrong and knowing why is the whole point of a lab.

Predict, then measure. Being wrong and knowing why is the whole point of a lab.

PRACTICAL · LIMITS

What This Lab Does Not Show You

- rtl_lint.py is regular expressions, not a Verilog parser. It reads code the way a careful reviewer skims it and can be fooled by unusual formatting.
- The cell counts come from Yosys' generic library, not a foundry library. They compare designs against each other; they are not area figures.
- fa_switch.v is a functional transistor model. There are no threshold voltages and no capacitances — SPICE is the tool for that.
- Yosys accepts initial blocks because it targets FPGAs. An ASIC flow would not, and that difference is a genuine trap.
- Vivado and ModelSim were not run. Every measured number here came from iverilog, ghdl and yosys, and is reproducible with make.
- Nothing here covers verification methodology beyond a directed testbench — that is Topic 5, and it is a large subject on its own.

SUMMARY

Topic 2 In Ten Lines

- RTL says which registers exist and what transfers into them. Nothing else.
- <= reads every right-hand side first, then updates together. = does not.
- Four levels of abstraction describe the same circuit — proved, not assumed.
- The behavioural description produced the smallest netlist: 5 cells against 6.
- Write at the highest level that says what you mean.
- Not all legal Verilog is synthesisable, and a / b cost 371 cells against 0.
- A combinational block that does not assign on every path builds a latch.
- An incomplete sensitivity list makes simulation and silicon disagree silently.
- Seven coding rules, checked by a tool, and cross-checked against Yosys.
- Verilog and VHDL produced identical transcripts over 18 cycles.

CLOSE

What To Do Next

By the end of this subtopic you can

- ✓ Read an always block and say which registers it creates, and which logic sits between them.
- ✓ Write the same function at four levels of abstraction, and say what each level costs you.
- ✓ Choose the right level - and defend the choice with a cell count.
- ✓ Prove two descriptions are the same circuit, instead of hoping.
- ✓ Say which Verilog constructs synthesise, which do not, and which are traps.
- ✓ Recognise an inferred latch before the tool builds one.
- ✓ Explain why an incomplete sensitivity list is worse than an error.
- ✓ Apply the seven coding rules, and say why each one exists.
- ✓ Read a VHDL design without needing it translated.
- ✓ Take a design from spec to formal proof and show your evidence at each stage.

And one habit to carry into every design you ever write

Before you write a line of RTL, know what you expect the tool to build. Then check whether it did.
Every bug in this topic lives in the gap between those two things.

Coming next in Module 2

Topic 3 covers the digital logic this all rests on. Topic 4 writes much more RTL. Topic 5 is verification — how you know it works. Topic 6 is timing — whether it is fast enough.